CSC3066 Deep Learning
Convolutional Neural Networks

Lu Bai (l.bai@qub.ac.uk)

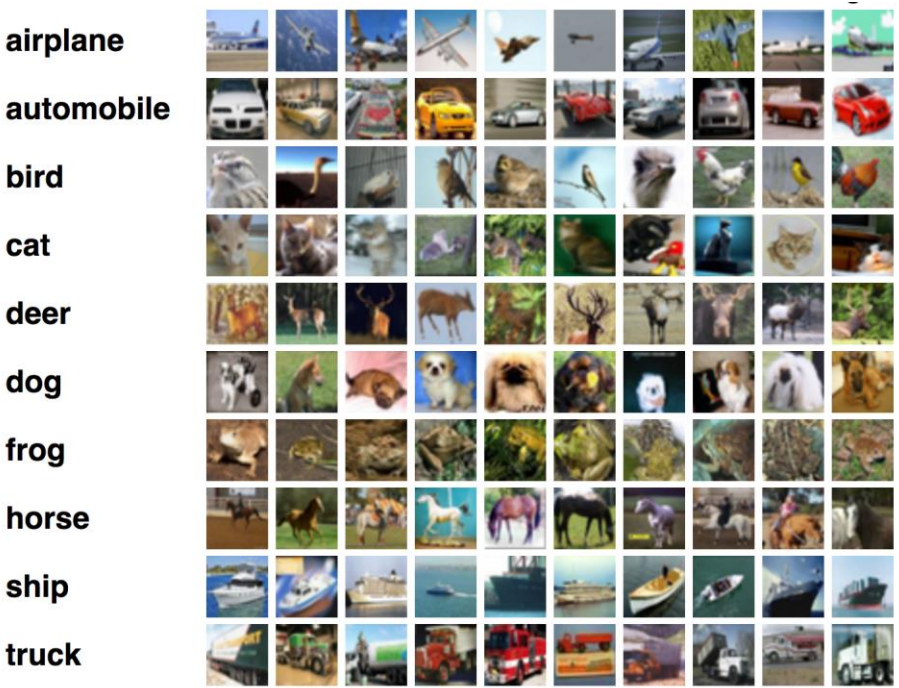
Content of Today Lecture

- Image Classification/Recognition
- Introduction to Convolutional Neural Networks (CNNs)
- CNNs and text data

Image Classification

- Image classification is the task of taking an input image and outputting a class (e.g. a cat, a dog,) or a probability of classes that best describes the image.
  - Medical image classification (labelling an x-ray as cancer or not)
  - Face/gender recognition (assigning a name to a photography of a face)
  - Recognising a hand written digits
  - …

Image Classification



Ref: https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html

Image Classification

- Interpretation of an image
  - Computers interpret images as arrays of pixels, each pixel in an image can be represented as a number (0-255).
  - When we start adding colours, each pixel is represented by 3 values (Red, Green and Blue).
  - The dimension of the pixel array depends on the resolution and size of the image (e.g. 32 x 32 x 3 array of numbers)

Image Classification

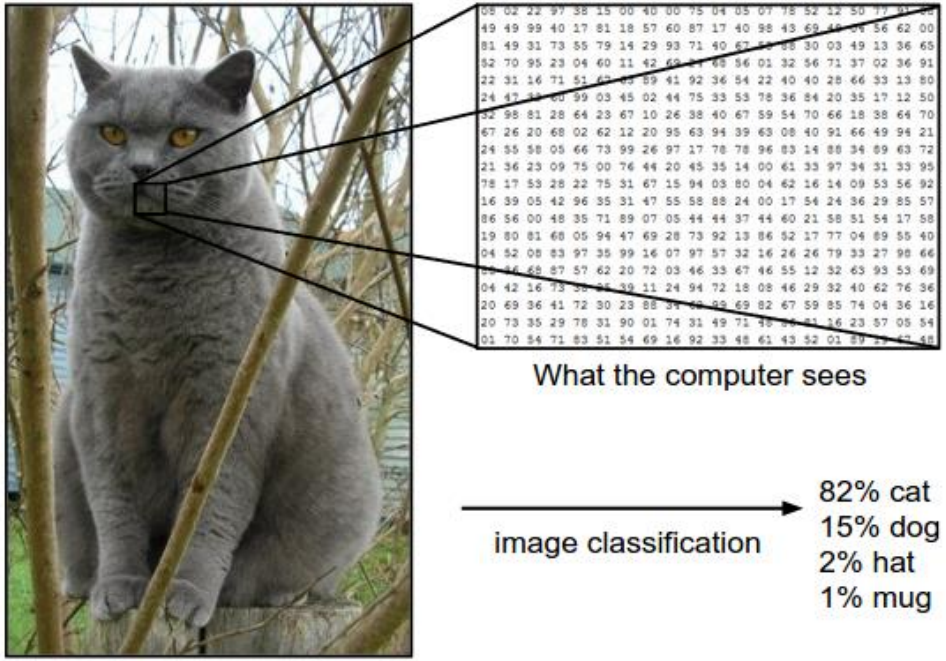
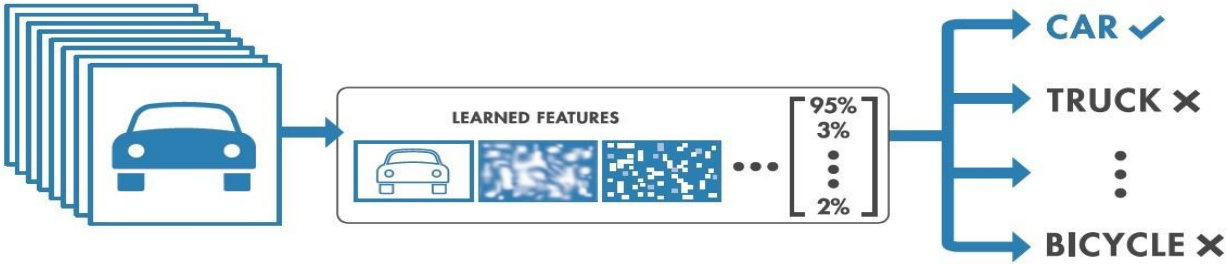


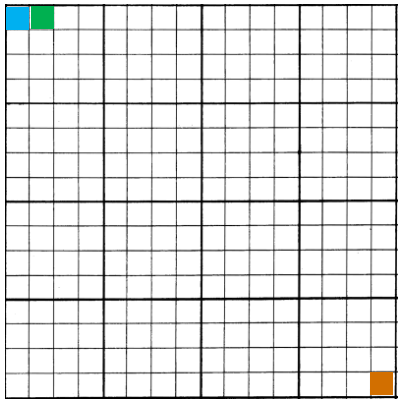
Image Classification

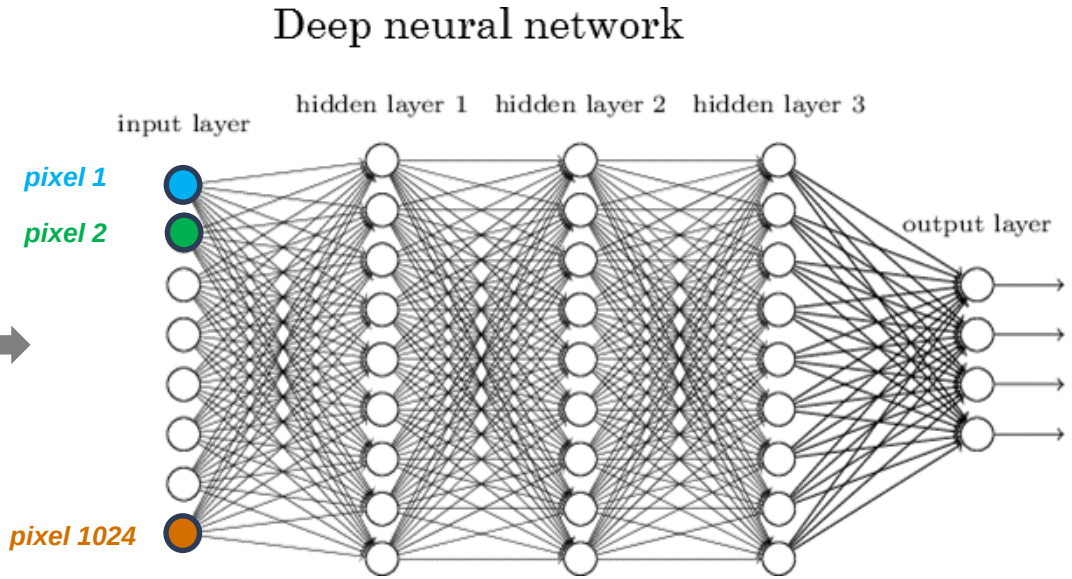
- Images and Machine Learning
  - The array of pixels, while meaningless to humans, can be used as an input to a machine learning model.
  - The idea is to train a machine learning model to recognize different patterns from an image based on its pixel representation.



The problem with MLP for image classification

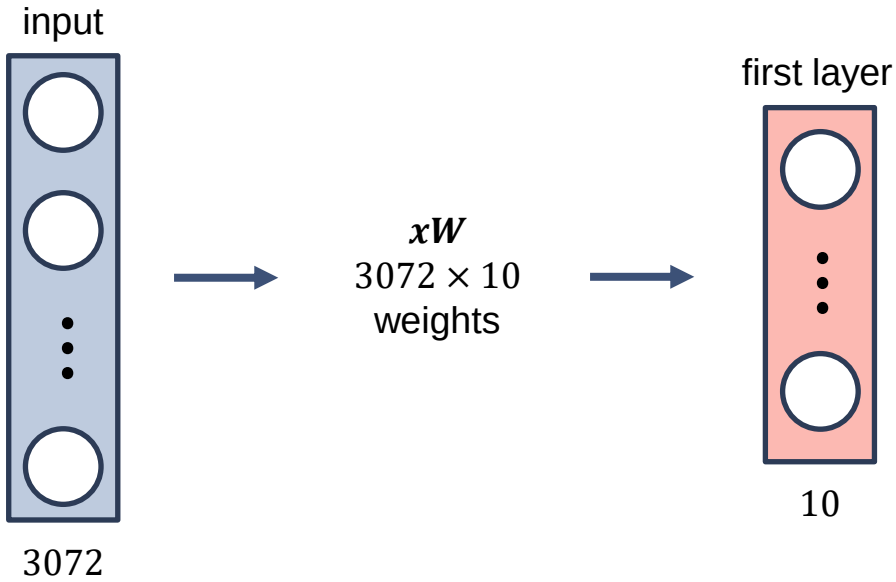
Input image (32×32)





The problem with MLP for image classification

- $32 \times 32 \times 3$ image will be stretched to 3072×1 vector

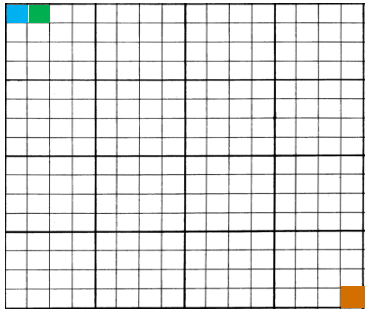


The problem with MLP for image classification

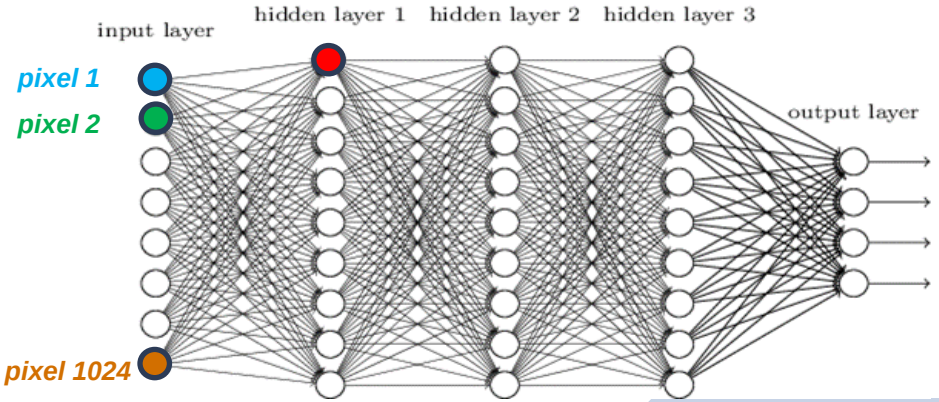
- Multilayer Perceptron and images
  - There are several drawbacks of MLP, especially when it comes to image processing.
  - MLP uses one input neuron for each input (e.g. pixel in an image, multiplied by 3 in RGB case).
  - The amount of weights rapidly becomes unmanageable for large images.
  - This can cause problems during training such as overfitting.

The problem with MLP for image classification

Input image



- It makes sense for the highlighted unit in hidden layer 1 to combine information from *pixel 1* and *pixel 2*: they may be part of the same local structure (e.g. a horizontal line).
- But does it really make sense to combine information from *pixel 1* and *pixel 1024*?



The problem with MLP for image classification

- Multilayer Perceptron and images
  - Beside the complexing, when we use MLP for image classification we lose the spatial information while the image is flattened (converted into a vector).
  - Nodes that are close together are important because they help to define the features of an image.
  - Clearly, MLP are not the best idea to use for image processing!

Convolutional Neural Network

Convolutional Neural Network

- Convolutional Neural Network (CNN)
  - CNN is the foundation of most computer vision technologies.
  - Unlike traditional MLP architectures, it uses two operations called convolution and pooling to reduce an image into its essential features
  - The identified features are further used to understand and classify an image.

Convolutional Neural Network

- A Convolutional Neural Network contains 4 different types of layers:
  - Convolutional Layer
  - Pooling Layer
  - Flatten Layer
  - Fully connected Layer

Convolutional Neural Network

Convolutional Layer

CNN: Convolutional Layers

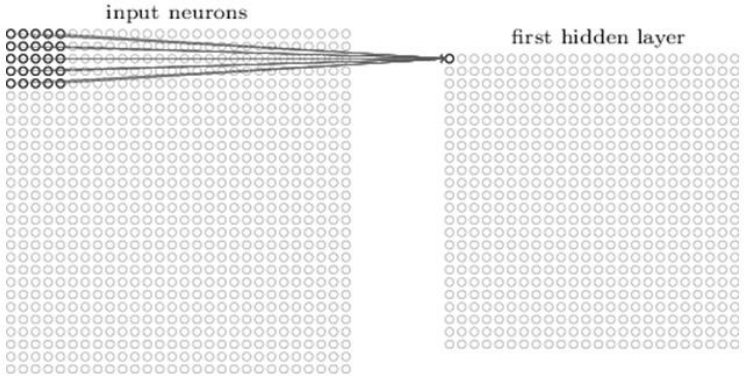
- Convolutional Layers
  - The most important building block of a CNN is the convolutional layer.
  - It is a partially connected layer that takes the local spatial structure of the data into account.
  - Neurons in the first convolutional layer are not connected to every single pixel in the input image, but only to pixels in their receptive fields.

CNN: Convolutional Layers

- Convolutional Layers
  - Each neuron in the second convolutional layer is connected only to neurons located within a small rectangle in the first layer.
  - This architecture allows the network to concentrate on low-level features in the first hidden layer, then assemble them into higher-level features in the next hidden layer, and so on.

CNN: Convolutional Layers

- Convolutional Layers
- If f_h and f_w are the height and width of the receptive field, a neuron at position (i, j) of a given layer is connected to the output of the neurons from the previous layer located in rows from i to $i + f_h - 1$ and columns j to $j + f_w - 1$.



CNN: Convolutional Layers

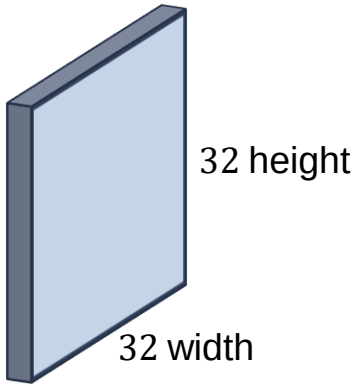
- Convolutional Layers
  - A neuron's weights are represented as an array of the size equals to the receptive field, called filters.
  - The depth of the filter has to be the same as the depth of the input volume.
  - The first position of the filter is in the top left corner of the input array.

CNN: Convolutional Layers

- Convolutional Layers
  - The filter is sliding, or convolving, across the image and it is multiplying the values in the filter with the original pixel values of the image (elementwise multiplication).
  - Those multiplications are all summed up resulting in a single value.
  - This operation is repeated for each position of the filter on the input image.

CNN: Convolutional Layers

- Convolutional Layers $32 \times 32 \times 3$ image example
  - Filter is a set of weights

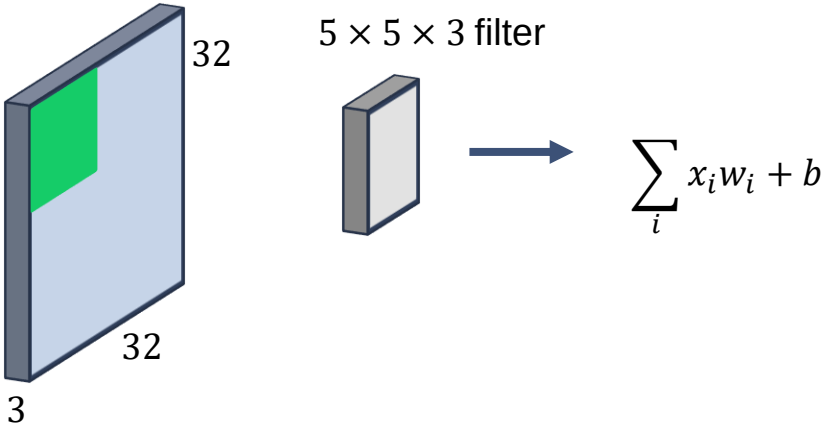


$5 \times 5 \times 3$ filter

Slide the filter over the image spatially computing dot product at every spatial location

CNN: Convolutional Layers

- Convolutional Layers
  - $32 \times 32 \times 3$ image example

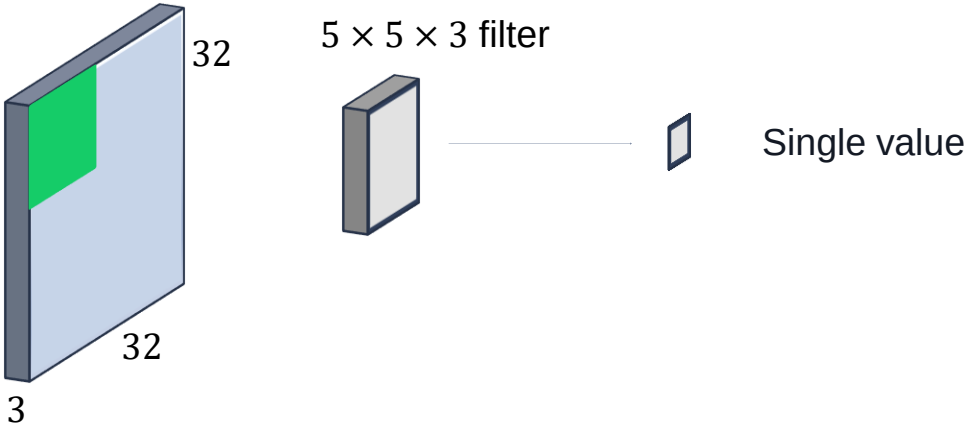


Single value: (dot product) elementwise multiplication between the weights defined in the filter and a $5 \times 5 \times 3$ chunk of the image (pixel values) plus bias (i.e. $5 \times 5 \times 3 = 75$-dimensional dot product + bias)

It's just a neuron with local connectivity…

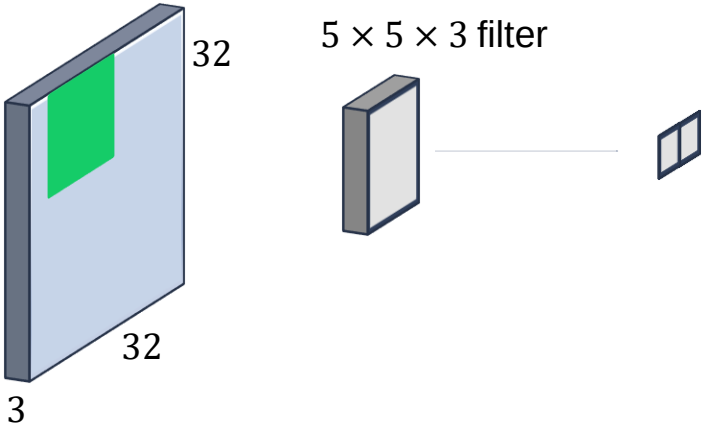
CNN: Convolutional Layers

- Convolutional Layers
  - $32 \times 32 \times 3$ image example
  - The filter is sliding across all spatial locations



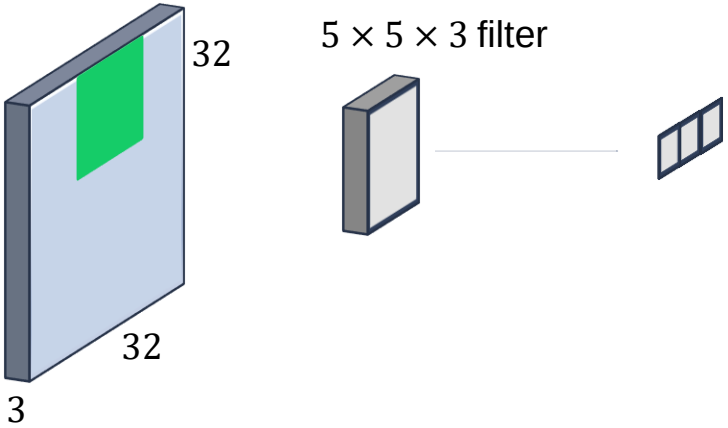
CNN: Convolutional Layers

- Convolutional Layers
  - $32 \times 32 \times 3$ image example
  - The filter is sliding across all spatial locations



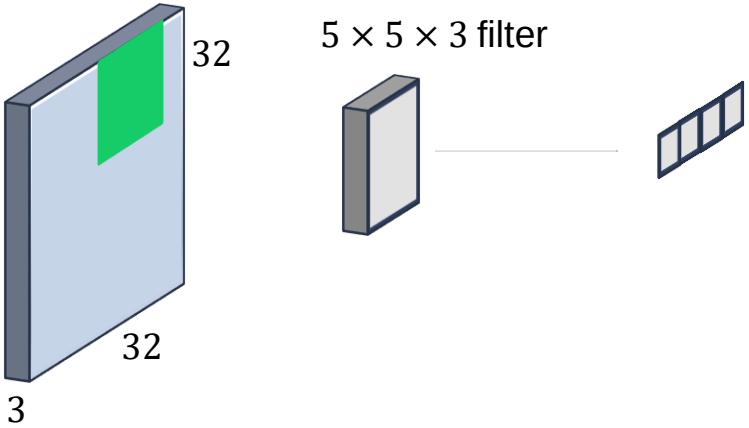
CNN: Convolutional Layers

- Convolutional Layers
  - $32 \times 32 \times 3$ image example
  - The filter is sliding across all spatial locations



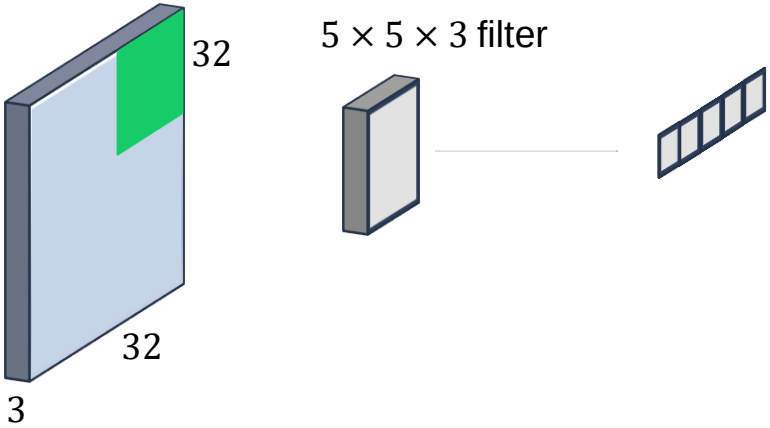
CNN: Convolutional Layers

- Convolutional Layers
  - $32 \times 32 \times 3$ image example
  - The filter is sliding across all spatial locations



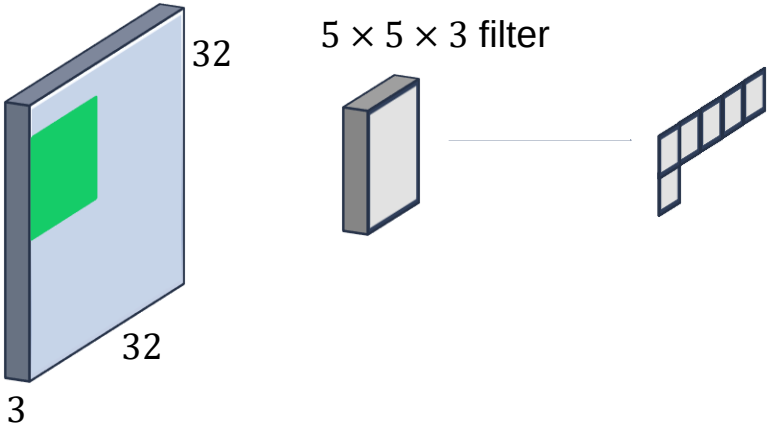
CNN: Convolutional Layers

- Convolutional Layers
  - $32 \times 32 \times 3$ image example
  - The filter is sliding across all spatial locations



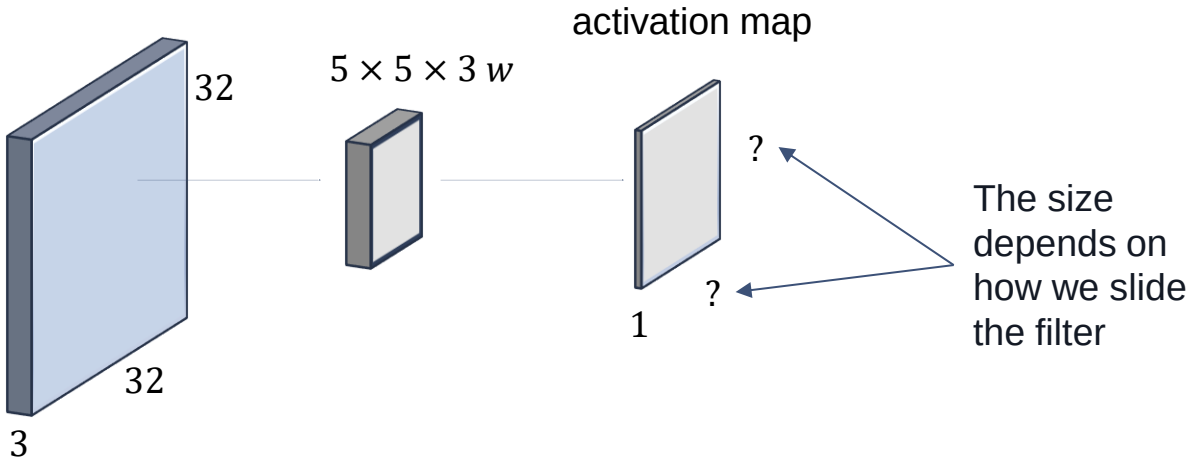
CNN: Convolutional Layers

- Convolutional Layers
  - $32 \times 32 \times 3$ image example
  - The filter is sliding across all spatial locations



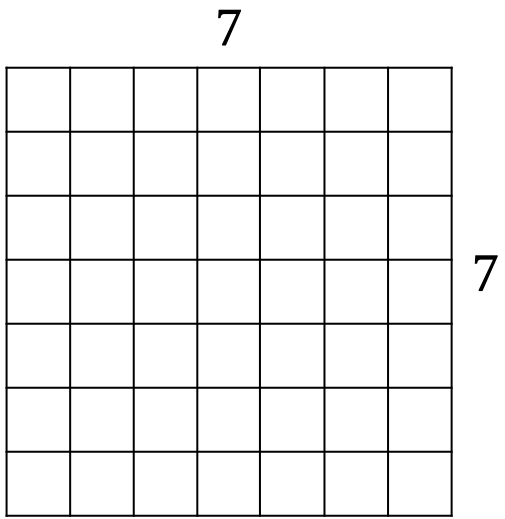
CNN: Convolutional Layers

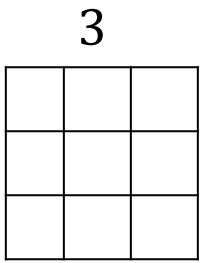
- Convolutional Layers
  - After sliding the filter over all the location, we obtain an array of numbers called **activation** or **feature map**.



CNN: Convolutional Layers

- Let's consider input as $7 \times 7 \times 1$ and filter $3 \times 3 \times 1$ for simplicity

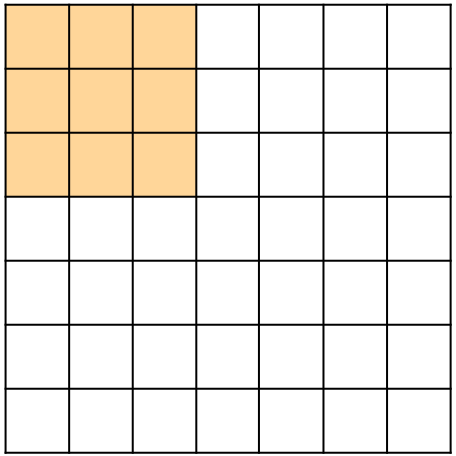




$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}$$

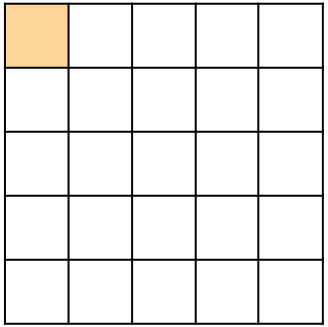
CNN: Convolutional Layers

- We apply the filter to the top left corner of the input



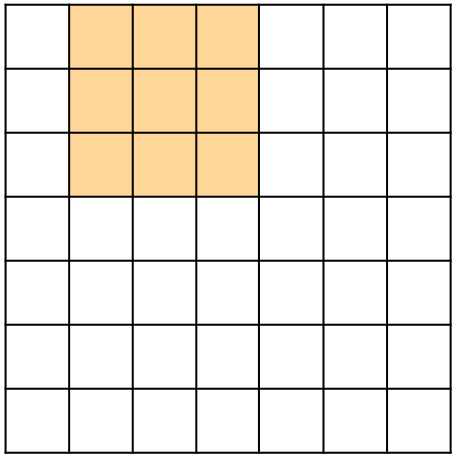
$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}$$

dot product



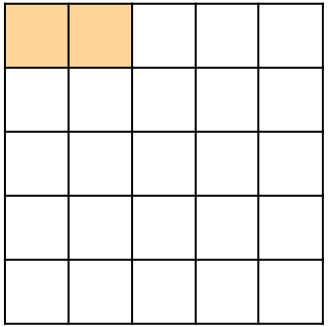
CNN: Convolutional Layers

- We slide the filter one position to the right



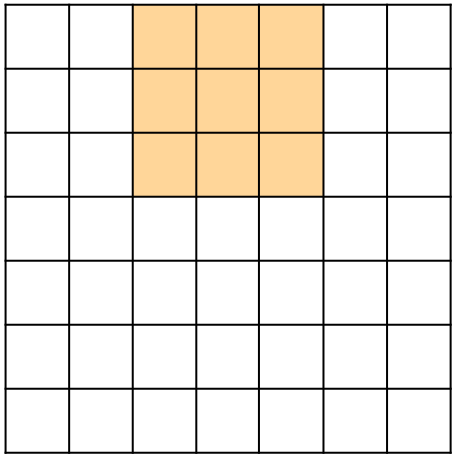
$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}$$

dot product



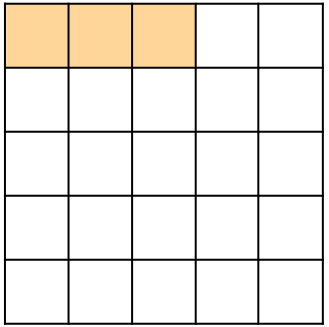
CNN: Convolutional Layers

- We slide the filter one position to the right



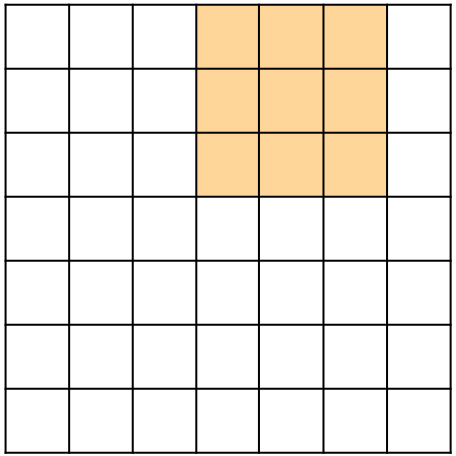
$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}$$

dot product



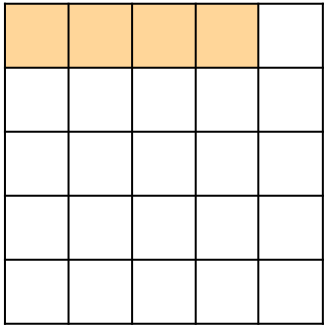
CNN: Convolutional Layers

- We slide the filter one position to the right



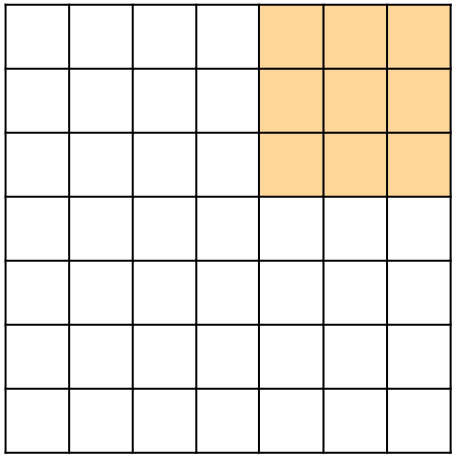
$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}$$

dot product



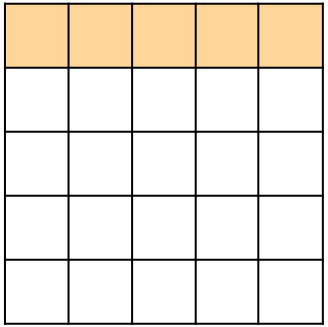
CNN: Convolutional Layers

- We slide the filter one position to the right



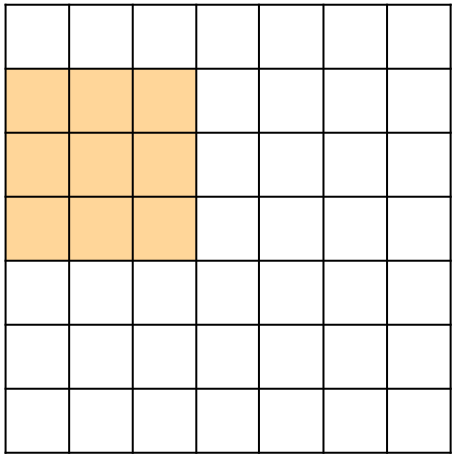
$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}$$

dot product



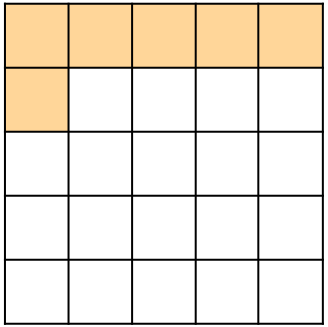
CNN: Convolutional Layers

- We move the filter back to the left side of the input and continue…



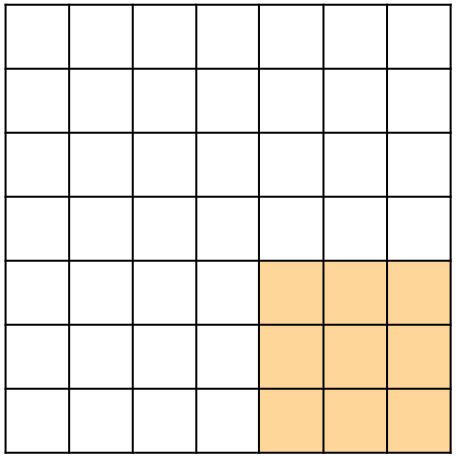
$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}$$

dot product



CNN: Convolutional Layers

- We were able to move the filter 5 spatial locations horizontally and 5 spatial location vertically



$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}$$

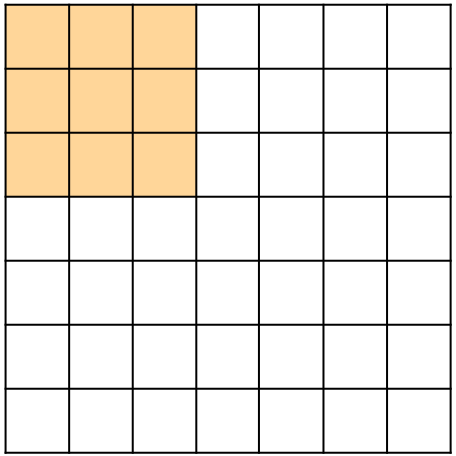
5×5 output

CNN: Convolutional Layers

- The interval which we use to slide the filter is called stride
- In the previous example we used stride $= 1$
- We can use different values of strides.

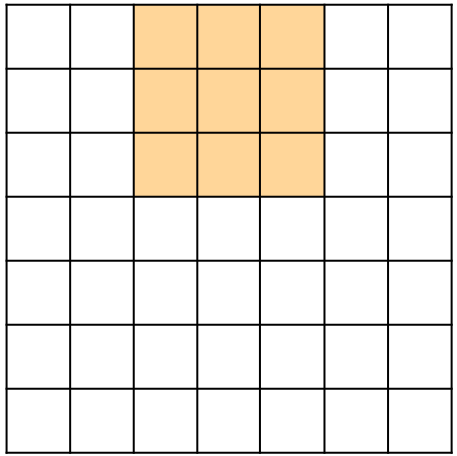
CNN: Convolutional Layers

- For stride = 2 we slide the filter two spatial locations to the right



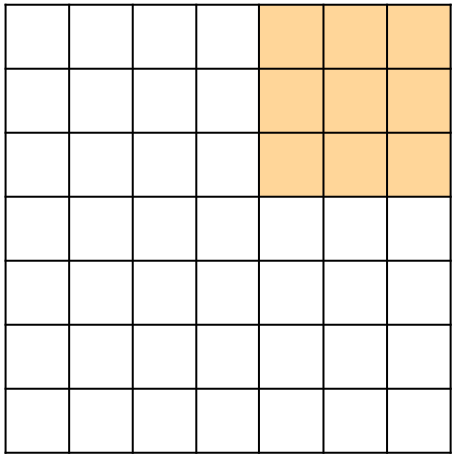
CNN: Convolutional Layers

- For stride = 2 we slide the filter two spatial locations to the right



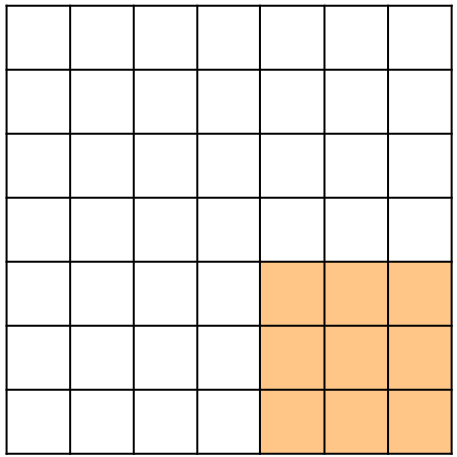
CNN: Convolutional Layers

- For stride = 2 we slide the filter two spatial locations to the right



CNN: Convolutional Layers

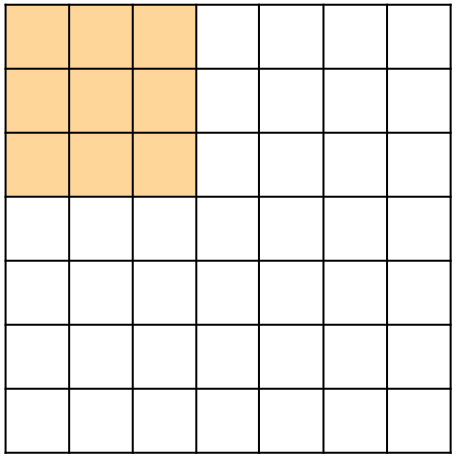
- For stride = 2 we slide the filter two spatial locations to the right



7×7 input
3×3 filter
stride 2
3×3 output

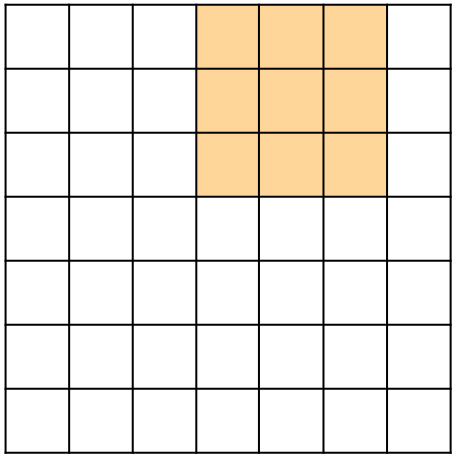
CNN: Convolutional Layers

- What happens for stride = 3?



CNN: Convolutional Layers

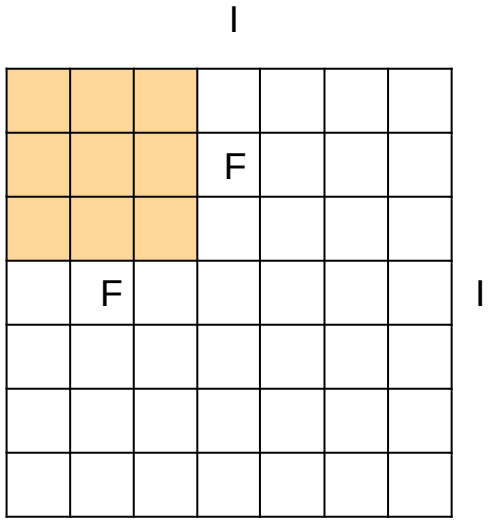
- What happens for stride = 3?



It doesn't fit!
We can't apply stride 3

CNN: Convolutional Layers

- Output size: $\frac{I-F}{stride}+1$



e.g. $I = 7, F = 3$:

Stride 1 $\rightarrow \frac{7-3}{1}+1=5$

Stride 2 $\rightarrow \frac{7-3}{2}+1=3$

Stride 3 $\rightarrow \frac{7-3}{3}+1=2.33$ This will not work

CNN: Convolutional Layers

- In order to make the size work it is common to zero pad the border of the image

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

CNN: Convolutional Layers

- In order to make the size work it is common to zero pad the border of the image

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Output size: $\frac{I-F}{stride} + 1$

What will be the dimension of the output now with filter of size 3 and stride 3?

CNN: Convolutional Layers

- What else is padding useful for in the convolving process?

0	0	0	0	0	0	0	0
0							0
0							0
0							0
0							0
0							0
0							0
0	0	0	0	0	0	0	0

CNN: Convolutional Layers

- Padding can also be used to stop the output from shrinking and loosing information on corners and edges of the image.

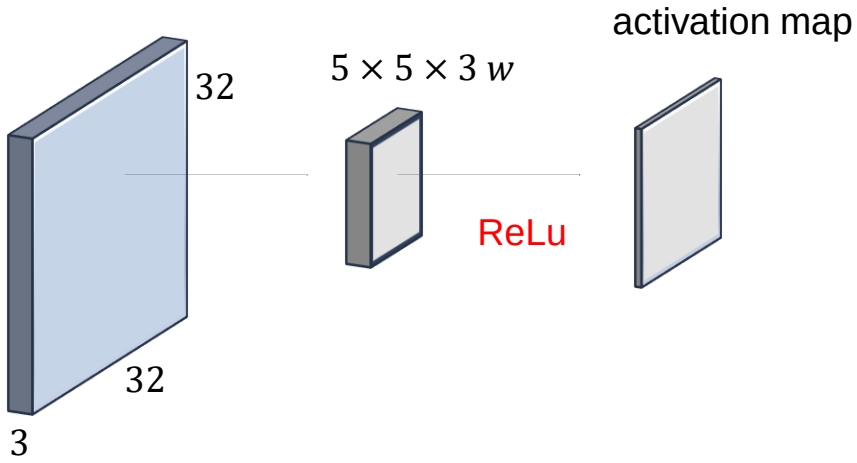
0	0	0	0	0	0	0	0
0							0
0							0
0							0
0							0
0							0
0							0
0	0	0	0	0	0	0	0

CNN: Convolutional Layers

- In order to allow for non-linearity, the values from the matrix generated in the convolutional layer are pass through an activation function.
- The activation function is typically ReLu

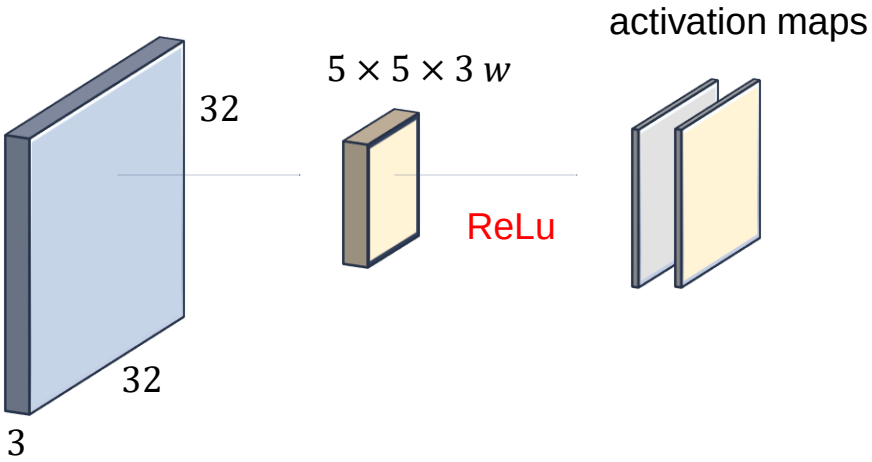
CNN: Convolutional Layers

- Convolution layer with ReLu activation function



CNN: Convolutional Layers

- In convolutional layer we can use multiple filters. Each filter looks for different features/shapes in the image (e.g. horizontal line)



CNN: Convolutional Layers

- We can have as many filters as we want in a convolutional layer

CNN: Convolutional Layers

- If we had 4 5×5 filters with stride 1, we will get 4 separate activation maps of size 28×28:

CNN: Convolutional Layers

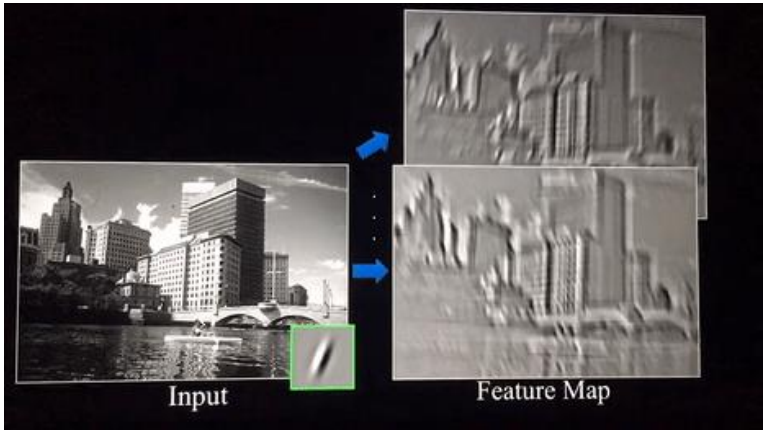
- Each activation map was computed with different set of parameters (weights and biases):

CNN: Convolutional Layers

- All the activation maps are stacked together and form a new image of size $28 \times 28 \times 4$

CNN: Convolutional Layers

- Convolutional layers are feature detectors
- Convolution of an image with different filters can perform operations such as edge detection, blur and sharpen an image by applying filters.



CNN: Convolutional Layers

- Summary of Convolutional Layer:
  - Accept an input of size $w_{in} \times h_{in} \times d_{in}$
  - Requires 4 parameters:
    - Number of filters: k
    - Filter size: f (can have different height and width $[f_h, f_w]$)
    - The stride: s
    - The amount of zero padding: p

CNN: Convolutional Layers

- Summary of Convolutional Layer:
  - Produces an output of size $w_{out} \times h_{out} \times d_{out}$ where:
    - $w_{out} = \frac{(w_{in} - f_w + 2p)}{s} + 1$
    - $h_{out} = \frac{(h_{in} - f_h + 2p)}{s} + 1$
    - $d_{out} = k$

CNN: Convolutional Layers

- Summary of Convolutional Layer
  - Common parameters settings:
    - k= power of 2, e.g. $32, 64, 128, 512$
    - $f = 3, s = 2, p = 1$
    - $f = 5, s = 1, p = 2$
    - $f = 5, s = 2, p =$? (whatever fits)
    - $f = 1, s = 1, p = 0$

CNN: Convolutional Layers

- Question 1:
  - Suppose we have 10 filters, each of shape $3 \times 3 \times 3$. What will be the number of the parameters in that layer?

CNN: Convolutional Layers

- Answer:
  - Suppose we have 10 filters, each of shape $3 \times 3 \times 3$. What will be the number of the parameters in that layer?
    - Number of parameters for each filter = $3 * 3 * 3 = 27$
    - There will be a bias term for each filter, so total parameters per filter = 28
    - As there are 10 filters, the total parameters for that layer = $28 * 10 = 280$

CNN: Convolutional Layers

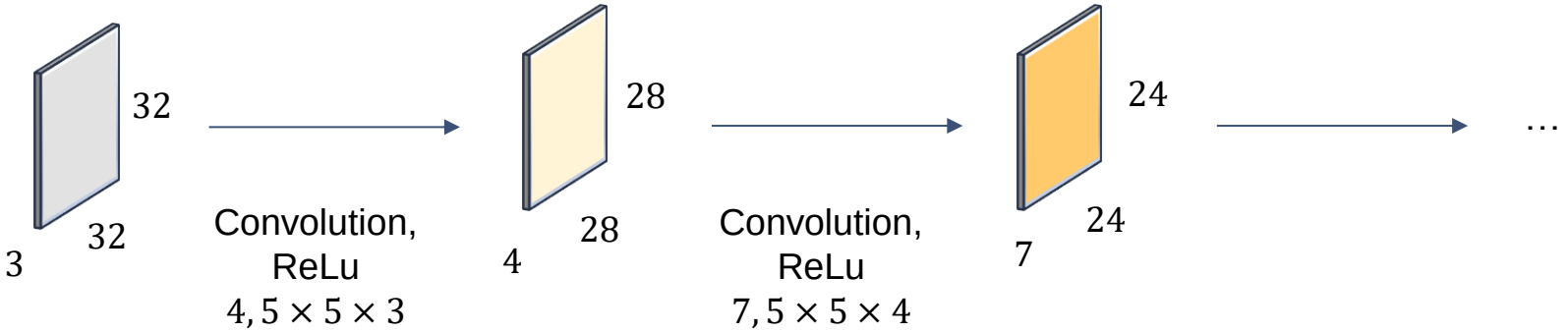
- Question 2:
  - Input volume: $32 \times 32 \times 3$
  - 10 $5 \times 5 \times 3$ filters with stride 1, pad 2
  - What will be the output volume size?

CNN: Convolutional Layers

- Answer:
  - Input volume: $32 \times 32 \times 3$
  - 10 5×5 filters with stride 1, pad 2
  - What will be the output volume size?
    - $\frac{(32-5+2*2)}{1} + 1 = 32$
    - $32 \times 32 \times 10$

CNN: Convolutional Layers

- Convolutional Neural Network is composed of a sequence of Convolutional Layers, interspersed with activation functions



CNN: Convolutional Layers

- Convolutional Layer in Keras:

```
from tensorflow import keras
from keras.layers import Conv2D
from keras.models import Sequential
```

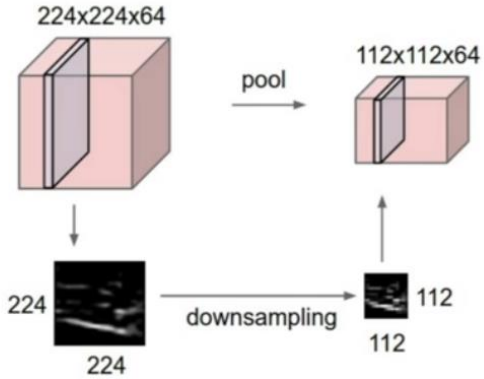
```
model = Sequential()
model.add(Conv2D(32, kernel_size=(3,3), strides=(1,1), activation='relu', padding='same', input_shape=(150,150,3)))
model.add(Conv2D(32, kernel_size=(3,3), strides=(3,3), activation='relu', padding='valid'))
```

Convolutional Neural Network

Pooling Layer

CNN: Pooling Layers

- Pooling layer: makes the representation smaller and more manageable (down sampling)
  - Operates on each activation map independently
  - Only affect the spatial dimensions, does not affect depth



CNN: Pooling Layers

- Pooling layer:
  - Generally applied to reduce the size of the input and speed up the computation.
  - Consider a 4×4 activation map (matrix)

4	5	6	4
7	6	1	4
1	2	0	1
1	1	9	2

CNN: Pooling Layers

- Max Pooling:
  - Applying max pooling on this matrix with 2×2 filter and stride = 2 will result in 2×2 output:

4	5	6	4
7	6	1	4
1	2	0	1
1	1	9	2

7	6
2	9

CNN: Pooling Layers

- Average Pooling:
  - Apart from max pooling, we can also apply average pooling where, instead of taking the max of the numbers, we take their average.

CNN: Pooling Layers

- Global Pooling:
  - Instead of down sampling patches of the input feature map, global pooling down samples the entire feature map to a single value.
  - Global pooling can be used in a model to aggressively summarize the presence of a feature in an image.
  - It is also sometimes used in models as an alternative to using a fully connected layer to transition from feature maps to an output prediction for the model.

CNN: Pooling Layers

- Summary of Pooling Layer:
  - Accept an input of size $w_{in} \times h_{in} \times d_{in}$
  - Requires 3 parameters:
    - Filter size: f
    - The stride: s
    - Max or average pooling

CNN: Pooling Layers

- Summary of Pooling Layer:
  - Produces an output of size $w_{out} \times h_{out} \times d_{out}$ where:
    - $w_{out} = \frac{(w_{in} - f_w + 2p)}{s} + 1$
    - $h_{out} = \frac{(h_{in} - f_h + 2p)}{s} + 1$
    - $d_{out} = d_{in}$

CNN: Pooling Layers

- Summary of Pooling Layer
  - Common parameters settings:
    - $f = 2, s = 2$
    - $f = 3, s = 2$

CNN: Pooling Layers

- Pooling Layer in Keras

```
from keras.layers import MaxPooling2D

model.add(MaxPooling2D(pool_size=(2,2), strides = None)) #strides=pool_size
```

Convolutional Neural Network

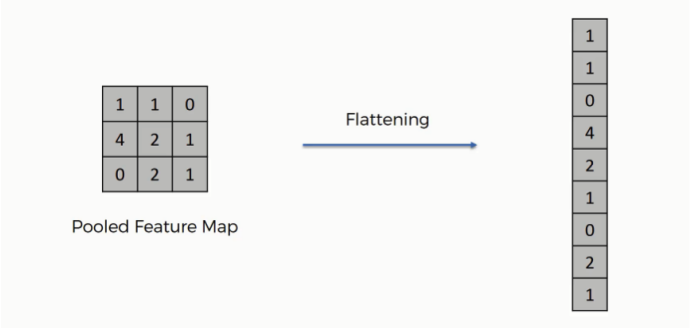
Flatten and Fully Connected Layers

CNN: Fully Connected Layer

- Fully Connected Layer
  - Traditional multilayer perceptron structure.
  - Its input is a one-dimensional vector representing the output of the previous layers.
  - Its output is a list of probabilities for different possible labels attached to the image (e.g. dog, cat, bird).
  - The label that receives the highest probability is the classification decision.

CNN: Flatten Layer

- Fully Connected Layer
  - In order to use fully connected layer, we need to flatten the feature maps (matrices) into vector.



CNN: Fully Connected Layers

- Fully Connected Layer in Keras

```
from keras.layers import Flatten, Dense

model.add(Flatten())
model.add(Dense(64, activation = 'relu'))
model.add(Dense(10, activation = 'softmax'))
```

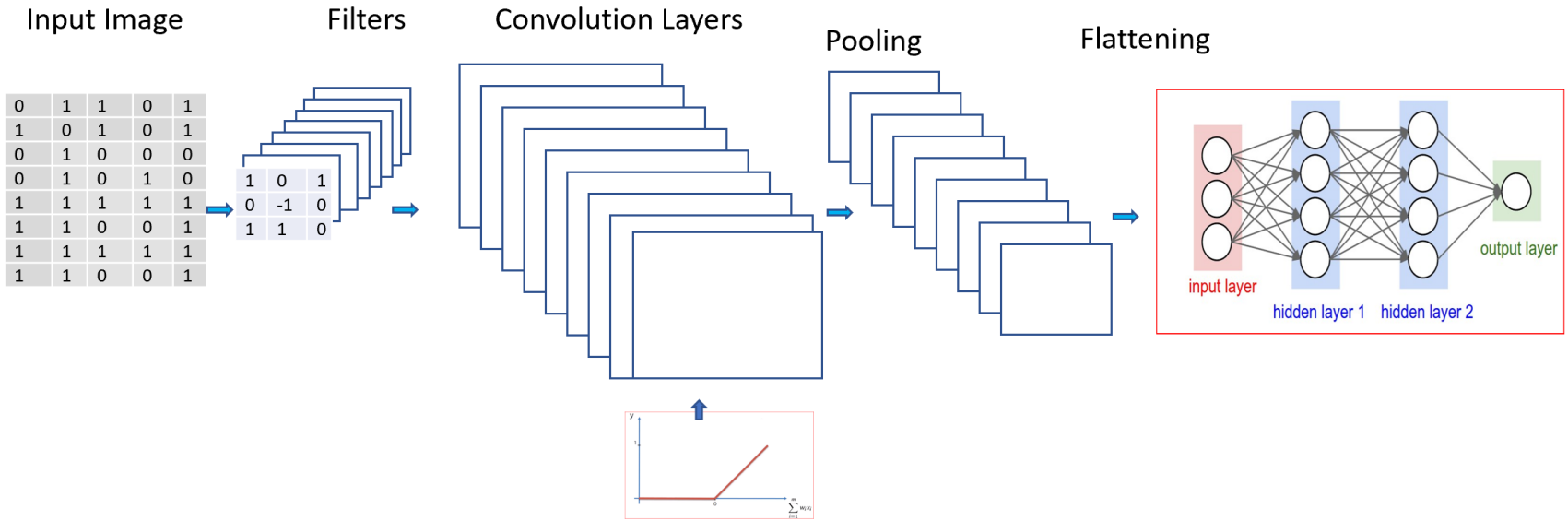
CNN: Fully Connected Layers

Model Summary

```
model.summary()
```

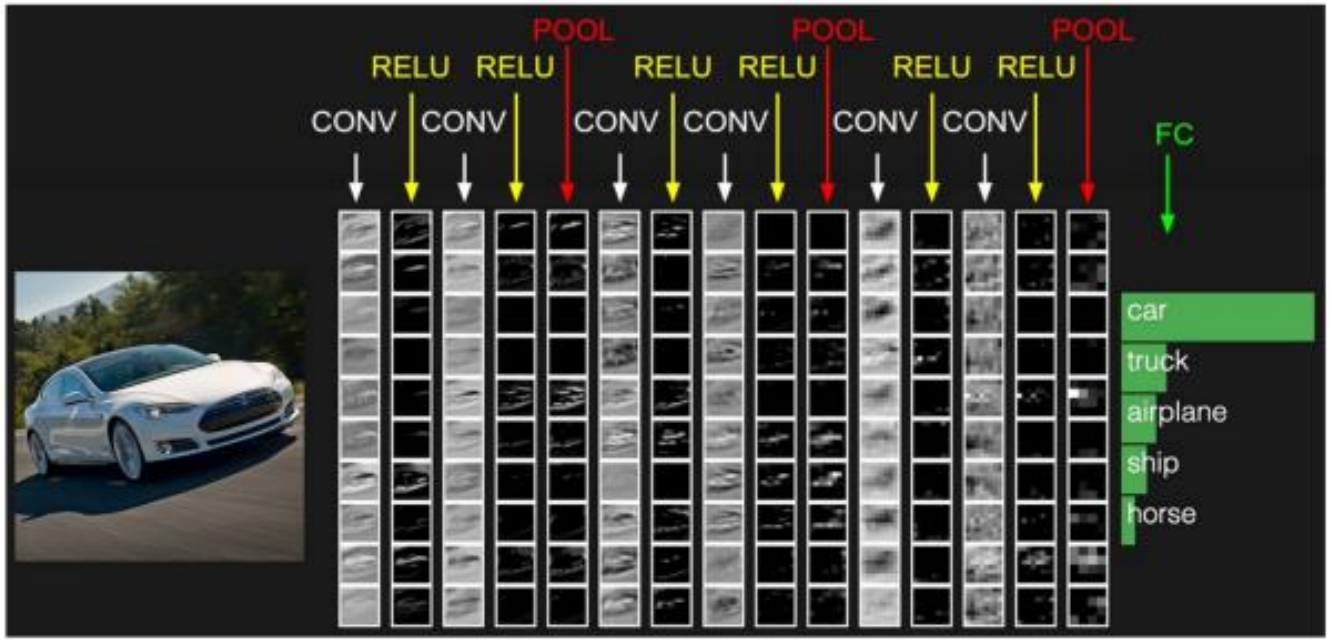
```
Model: "sequential_3"
_________________________________________________________________
 Layer (type)                 Output Shape              Param #
=================================================================
 conv2d_6 (Conv2D)            (None, 150, 150, 32)      896

 conv2d_7 (Conv2D)            (None, 50, 50, 32)        9248

 max_pooling2d_3 (MaxPooling  (None, 25, 25, 32)       0
 2D)

 flatten_1 (Flatten)          (None, 20000)             0

 dense_2 (Dense)              (None, 64)                1280064

 dense_3 (Dense)              (None, 10)                650

=================================================================
```

CNN: Putting It All Together



CNN: Putting It All Together

- Visualising each layer in CNN



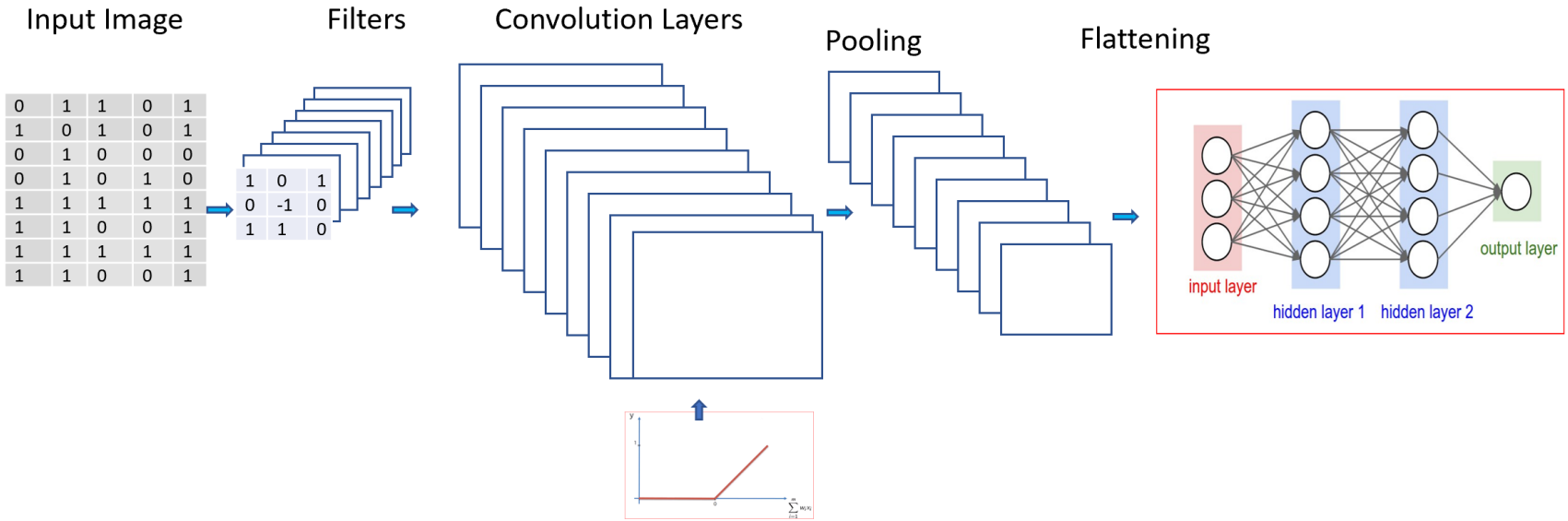
CNN

Working with text data

Convolutional Neural Network

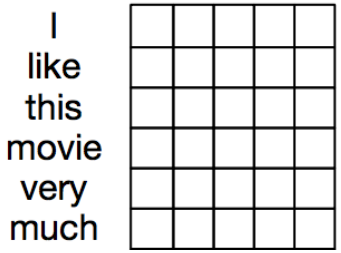
- A Convolutional Neural Network contains 4 different types of layers:
  - Convolutional Layer
  - Pooling Layer
  - Flatten Layer
  - Fully connected Layer

CNN: Putting It All Together



CNN for Text Classification

- Convolutional layers can be applied to text data represented with word embeddings:



CNN for Text Classification

- Convolutional Layer for Text
  - The filter will still be sliding over the input array but this time it will be looking at embeddings of multiple words, rather than a small area of pixels in an image.
  - To look at sequences of word embeddings, the kernels will no longer be squared.
  - Instead, they will be rectangles with dimensions:

$$embedding\ num \times embedding\ dim$$

CNN for Text Classification

- Convolutional Layer for Text
  - The height of the kernel is the number of embeddings we want to see at once (similar to n-grams)
  - The width of the kernel should span the length of the word embedding

Length of embeddings

-0.1	0.6	-0.7
0.2	0.5	0.1

Num of words to look at in sequence

CNN for Text Classification

- Convolutional Layer for Text
  - Toy example: 2-gram kernel with embeddings of size 5
  -

Word embedding

I	0.1	-0.3	0.6	-0.4	0.8
like	-0.7	-0.2	0.3	-0.3	0.5
this	-0.2	-0.3	-0.4	-0.2	0.3
movie	0.1	0.8	0.5	0.9	-0.2
very	0.6	0.9	0.9	-0.9	0.1
much	0.3	0.1	-0.3	0.7	0.8

Filter

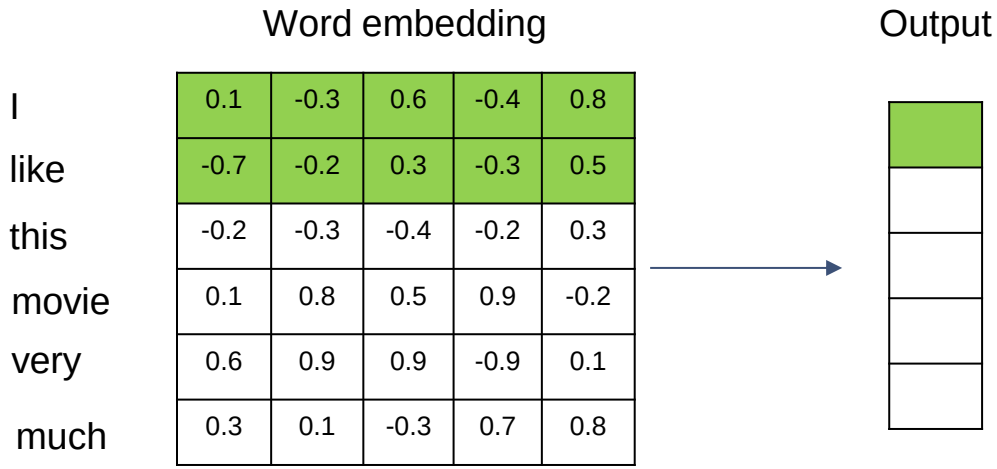
-0.1	-0.6	-0.6	-0.4	0.8
0.7	0.2	-0.3	-0.3	0.5

CNN for Text Classification

- Convolutional Layer for Text
  - To process a sequence of words, filter will slide down the list of word embeddings in sequence.
  - This is called 1D convolution because the filter is moving in only one dimension (time).
  - The result output of this operation will be a feature vector with size depending on the input and the filter size

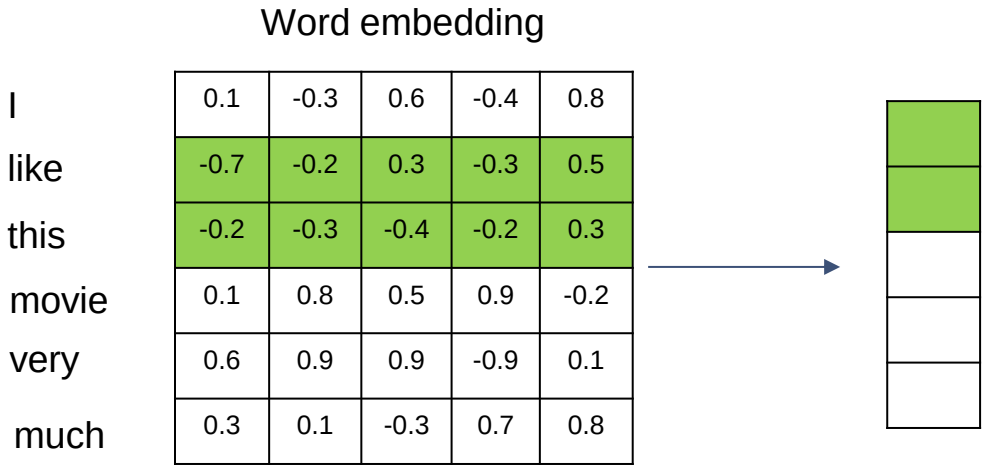
CNN for Text Classification

- Convolutional Layer for Text
  - Toy example: 2-gram kernel with embeddings of size 5



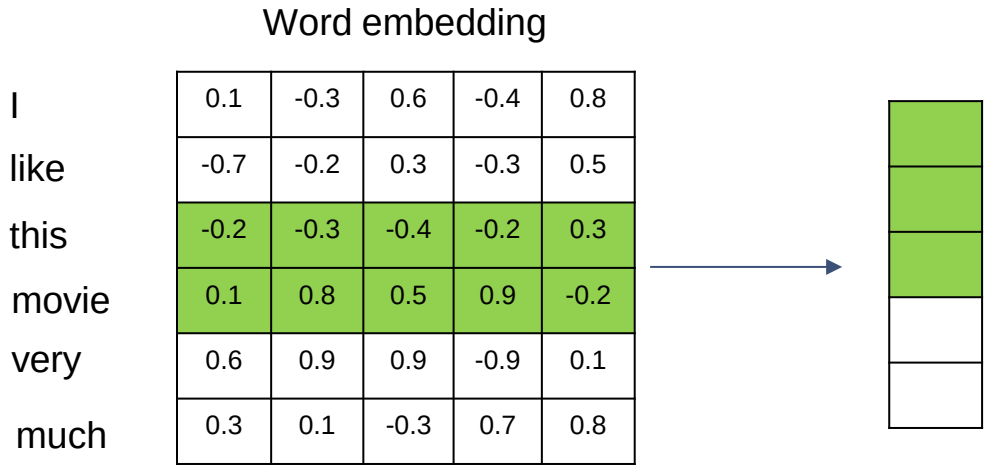
CNN for Text Classification

- Convolutional Layer for Text
  - Toy example: 2-gram kernel with embeddings of size 5



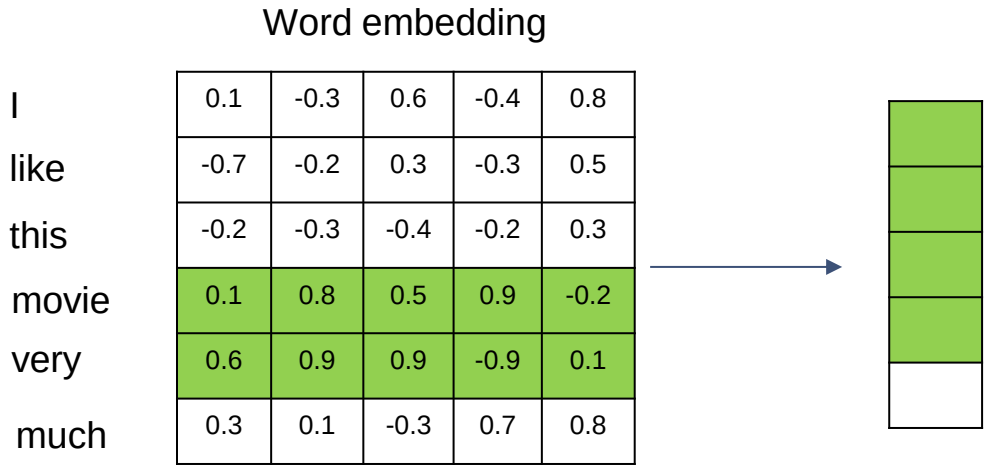
CNN for Text Classification

- Convolutional Layer for Text
  - Toy example: 2-gram kernel with embeddings of size 5



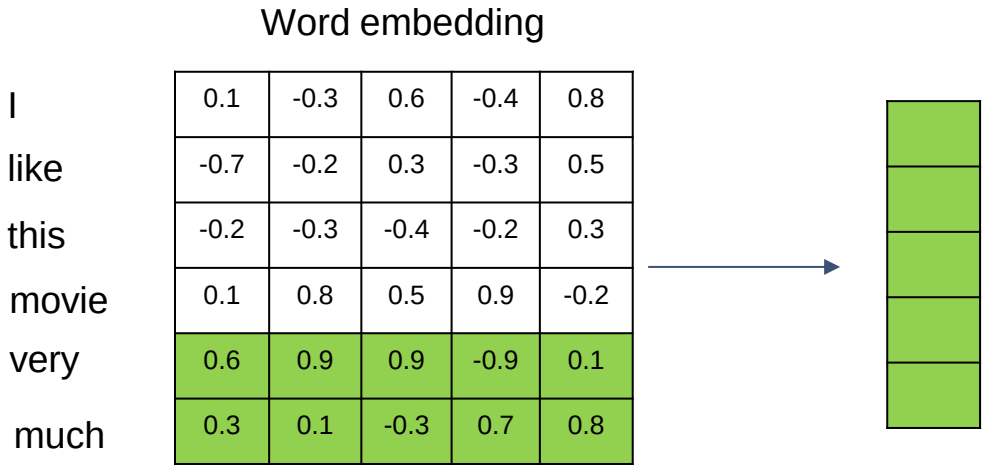
CNN for Text Classification

- Convolutional Layer for Text
  - Toy example: 2-gram kernel with embeddings of size 5



CNN for Text Classification

- Convolutional Layer for Text
  - Toy example: 2-gram kernel with embeddings of size 5



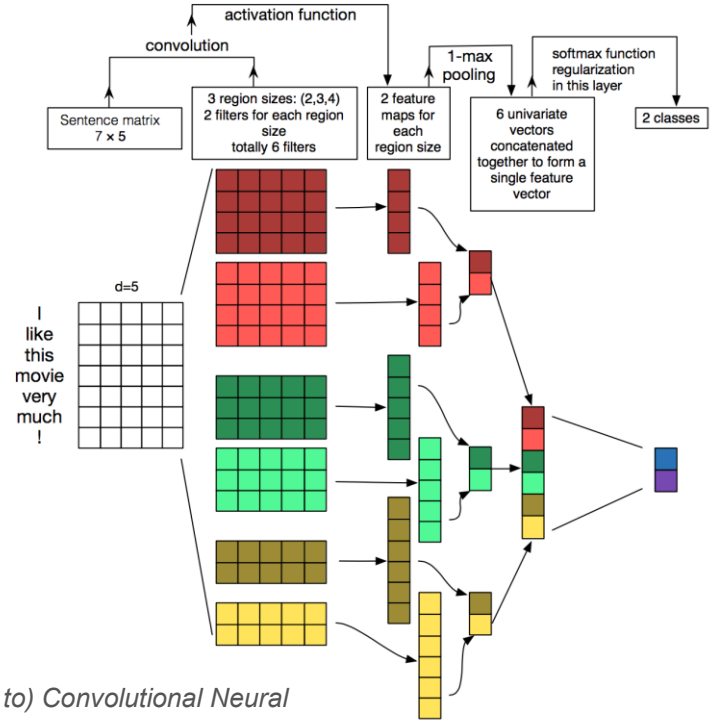
CNN for Text Classification

- Convolutional Layer for Text
  - Convolutional layers can be looked at as feature extraction (patterns in sequential words groupings that indicate traits)
  - CNN will contain multiple filters. One filter may not be not enough to detect different type of dependencies that are useful for a text classification task.
  - Similar words have similar embeddings, therefore, when a filter is applied to different sets of similar words, it will produce a similar output value.

CNN for Text Classification

- Pooling Layer for Text
  - The output of a convolutional layer is a feature vector
  - Polling layer can be applied to summarise the feature vector
  - It also helps to discard location information, which may not be important in the classification task (e.g. if in a movie review we see a phrase “great plot”, it doesn’t matter where we see it)

CNN for Text Classification



"*A Sensitivity Analysis of (and Practitioners' Guide to) Convolutional Neural Networks for Sentence Classification*", 2015.

CNN with Text

Implementation

CNN with Text: Implementation

- Architecture:
  - An embedding layer
  - A convolutional layer (Conv1D)
  - Max pooling layer
  - Fully connected output layers

CNN with Text: Implementation

- Data preparation:
  - The input data has to be integer encoded, so that each word is represented by a unique integer.
  - This data preparation step can be performed using the Tokenizer API also provided with Keras.
  - You can specify the vocabulary size via the num_words parameter. The tokenizer will pick the num_words of the most common words from your train dataset.

CNN with Text: Implementation

- Data preparation:

```
from keras.preprocessing.text import Tokenizer
tokenizer = Tokenizer(num_words=50000)
tokenizer.fit_on_texts(x_train)
sequences = tokenizer.texts_to_sequences(x_train)
```

CNN with Text: Implementation

- Data preparation:

```
for x in x_train[:3]:
    print(x)
```

```
I love this place.
The restaurant atmosphere was exquisite.
Google mediocre and I imagine Smashburger will pop up.
```

```
sequences[:3]
```

```
[[3, 93, 8, 14], [1, 50, 145, 4, 696], [697, 229, 2, 3, 698, 699, 42, 700, 60]]
```

CNN with Text: Implementation

- Data preparation:
  - The input data has to be of the same lengths (different documents/sentences have different number of words)
  - This can be addressed using padding (adding zeros to the sequences to make them the same length)
  - We can do this with a built $pad\_sequences()$ function in Keras
  - Find the max length of a document and use the function to convert all sequences from the corpus into the same length.

CNN with Text: Implementation

- Data preparation:

```
from keras_preprocessing.sequence import pad_sequences
x_train_seq = pad_sequences(sequences, maxlen=max_length)
print(x_train_seq.shape)
x_train_seq[:3]
```

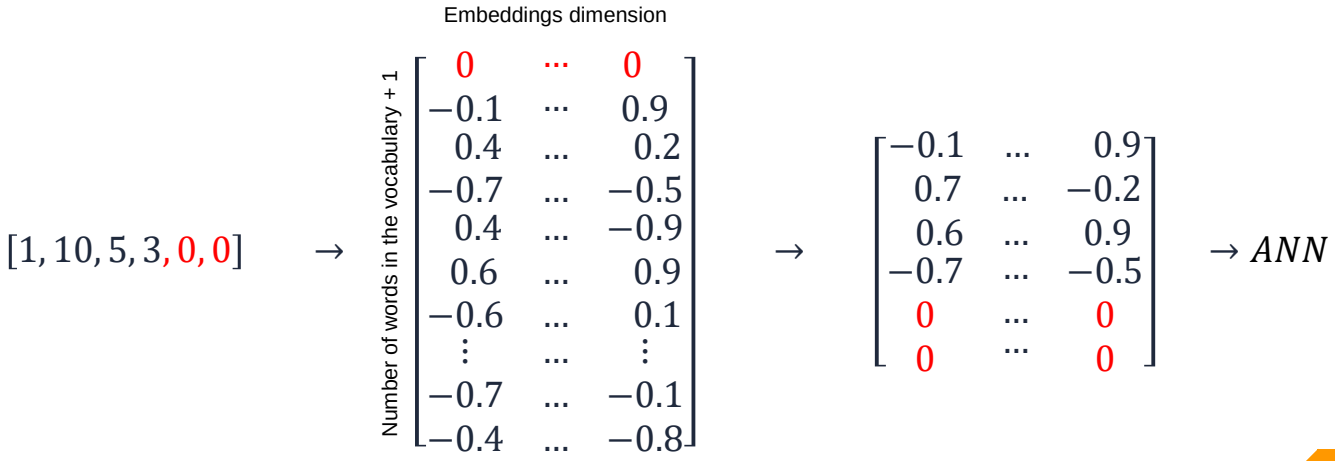
```
(697, 42)

array([[  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   3,
         93,   8,  14],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   1,  50,
        145,   4, 696],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0, 697, 229,   2,   3, 698, 699,
         42, 700,  60]], dtype=int32)
```

CNN with Text: Implementation

- Architecture:
  - Embedding layer:
    - A simple lookup table that stores embeddings of a fixed dictionary and size.
    - This module is used to store word embeddings and retrieve them using indices.
    - The input to the module is a list of indices, and the output is the corresponding word embeddings.

CNN with Text: Implementation

- Architecture:
  - Embedding layer:



Input list of indices (words) Lookup table Embeddings of the input words

CNN with Text: Implementation

- Architecture:
  - Embedding layer:
    - It can only be used as the first layer in a model
    - It converts word tokens (encoded as integers) into embedded vectors of a specific size.
    - The vectors/weights of this layer will come from a pretrained lookup table or can be trained as a part of the model training process.

CNN with Text: Implementation

- Architecture:
  - Embedding layer:
    - You can use the Embedding Layer of Keras which takes the previously calculated integers and maps them to a dense vector of the embedding.
    - You will need the following parameters:
      - input_dim: the size of the vocabulary
      - output_dim: the size of the dense vector
      - input_length: the length of the sequence

CNN with Text: Implementation

- Architecture:
  - Embedding layer:
    - The weights of the embedding layer can be initialized with random values and adjusted through backpropagation during training.
    - Another option is to use pre-trained word embeddings (e.g. w2v)
      - For this purpose we need to create an embedding matrix (weights), pass it to the embedding layer and set these parameters as not trainable.

CNN with Text: Implementation

- Architecture:
  - Embedding layer:

```
cnn = Sequential()
cnn.add(Embedding(num_words, 300, weights=[embedding_matrix], input_length=40, trainable=False))
cnn.add(Conv1D(128, 5, activation='relu', kernel_initializer=init_he_u))
cnn.add(GlobalMaxPooling1D())
cnn.add(Dense(10, activation='relu'))
cnn.add(Dense(1, activation='sigmoid'))
cnn.compile(loss='binary_crossentropy', optimizer='adam', metrics=['acc'])
cnn.fit(x_train_seq, y_train, validation_data=(x_test_seq, y_test), epochs=30, batch_size=32, verbose=2)
```

Coming Next

Recurrent Neural Networks